



AIRLINE DELAY CAUSES

Made by:

- Irvan Ilahibaks - 8081847
- Timon Baris - 2173182
- Waling van Dongen - 1176226

TA: Sita Newer

Vak: Data Analytics

28-10-2024

Task 1 - (level 2)

Importing data

```
1 import pandas as pd
2
3 dfData = pd.read_csv('airlinedelaycauses_DelayedFlights.csv')
```

Task 1 - (level 2)

Exploring data

```
1 dfData.info() |  
  
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 1936758 entries, 0 to 1936757  
Data columns (total 30 columns):  
#   Column              Dtype  
---  ---  
0   Unnamed: 0          int64  
1   Year                int64  
2   Month              int64  
3   DayOfMonth         int64  
4   DayOfWeek          int64  
5   DepTime            float64  
6   CRSDepTime         int64  
7   ArrTime            float64  
8   CRSArrTime         int64  
9   UniqueCarrier      object  
10  FlightNum          int64  
11  TailNum            object  
12  ActualElapsedTime  float64  
13  CRSElapsedTime     float64  
14  AirTime            float64  
15  ArrDelay           float64  
16  DepDelay           float64  
17  Origin             object  
18  Dest              object  
19  Distance           int64  
20  TaxiIn            float64  
21  TaxiOut           float64  
22  Cancelled         int64  
23  CancellationCode  object  
24  Diverted          int64  
25  CarrierDelay      float64  
26  WeatherDelay      float64  
27  NASDelay          float64  
28  SecurityDelay     float64  
29  LateAircraftDelay float64  
dtypes: float64(14), int64(11), object(5)  
memory usage: 443.3+ MB
```

```
1 dfData.describe() |
```

	FlightNum	...	Distance	TaxiIn	TaxiOut	Cancelled	Diverted	CarrierDelay	WeatherDelay	NASDelay	SecurityDelay	LateAircraftDelay
1758e+08	...	1.936758e+08	1.929648e+08	1.936303e+08	1.936758e+08	1.936758e+08	1.247488e+08	1.247488e+08	1.247488e+08	1.247488e+08	1.247488e+08	1.247488e+08
1263e+03	...	7.656882e+02	6.812975e+00	1.823220e+01	3.268348e-04	4.003598e-03	1.917940e+01	3.703571e+00	1.502184e+01	9.013714e-02	2.529847e+01	2.529847e+01
1702e+03	...	5.744797e+02	5.273595e+00	1.433853e+01	1.807562e-02	6.314722e-02	4.354821e+01	2.149290e+01	3.383305e+01	2.022714e+00	4.205486e+01	4.205486e+01
1000e+00	...	1.100000e+01	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
1000e+02	...	3.380000e+02	4.000000e+00	1.000000e+01	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
1000e+03	...	6.080000e+02	6.000000e+00	1.400000e+01	0.000000e+00	0.000000e+00	2.000000e+00	0.000000e+00	2.000000e+00	0.000000e+00	8.000000e+00	8.000000e+00
1000e+03	...	9.980000e+02	8.000000e+00	2.100000e+01	0.000000e+00	0.000000e+00	2.100000e+01	0.000000e+00	1.500000e+01	0.000000e+00	3.300000e+01	3.300000e+01
1000e+03	...	4.962000e+03	2.400000e+02	4.220000e+02	1.000000e+00	1.000000e+00	2.436000e+03	1.352000e+03	1.357000e+03	3.920000e+02	1.318000e+03	1.318000e+03

Task 1 - (level 2)

Exploring data

```
1 print(dfData.to_string)
```

<bound method DataFrame.to_string of Unnamed: 0 Year Month DayOfMonth DayOfWeek DepTime CRSDepTime \

0	0	2008	1	3	4	2003.0	1955							
1	1	2008	1	3	4	754.0	735							
2	2	2008	1	3	4	628.0	620							
3	4	2008	1	3	4	1829.0	1755							
4	5	2008	1	3	4	1940.0	1915							
...	...	...	...	...	...	...	...							
1936753	7009710	2008	12	13	6	1250.0	1220							
1936754	7009717	2008	12	13	6	657.0	600							
1936755	7009718	2008	12	13	6	1007.0	847							
1936756	7009726	2008	12	13	6	1251.0	1240							
1936757	7009727	2008	12	13	6	1110.0	1103							

ArrTime CRSArrTime UniqueCarrier ... TaxiIn TaxiOut Cancelled \

0	2211.0	2225	WN	...	4.0	8.0	0
1	1002.0	1000	WN	...	5.0	10.0	0
2	804.0	750	WN	...	3.0	17.0	0
3	1959.0	1925	WN	...	3.0	10.0	0
4	2121.0	2110	WN	...	4.0	10.0	0
...	...	...	...	...	...	...	...
1936753	1617.0	1552	DL	...	9.0	18.0	0
1936754	904.0	749	DL	...	15.0	34.0	0
1936755	1149.0	1010	DL	...	8.0	32.0	0
1936756	1446.0	1437	DL	...	13.0	13.0	0
1936757	1413.0	1418	DL	...	8.0	11.0	0

CancellationCode Diverted CarrierDelay WeatherDelay NASDelay \

0	N	0	NaN	NaN	NaN
1	N	0	NaN	NaN	NaN
2	N	0	NaN	NaN	NaN
3	N	0	2.0	0.0	0.0
4	N	0	NaN	NaN	NaN
...	...	...	...	...	...
1936753	N	0	3.0	0.0	0.0
1936754	N	0	0.0	57.0	18.0
1936755	N	0	1.0	0.0	19.0
1936756	N	0	NaN	NaN	NaN
1936757	N	0	NaN	NaN	NaN

SecurityDelay LateAircraftDelay

0	NaN	NaN
1	NaN	NaN
2	NaN	NaN
3	0.0	32.0
4	NaN	NaN
...	...	...
1936753	0.0	22.0
1936754	0.0	0.0
1936755	0.0	79.0
1936756	NaN	NaN
1936757	NaN	NaN

```
1 dfData.head() #show the first 5 rows
```

Unnamed: 0	Year	Month	DayOfMonth	DayOfWeek	DepTime	CRSDepTime	ArrTime	CRSArrTime	UniqueCarrier	...	TaxiIn	TaxiOut	Cancelled	Cancellation
0	0	2008	1	3	4	2003.0	1955	2211.0	2225	WN	...	4.0	8.0	0
1	1	2008	1	3	4	754.0	735	1002.0	1000	WN	...	5.0	10.0	0
2	2	2008	1	3	4	628.0	620	804.0	750	WN	...	3.0	17.0	0
3	4	2008	1	3	4	1829.0	1755	1959.0	1925	WN	...	3.0	10.0	0
4	5	2008	1	3	4	1940.0	1915	2121.0	2110	WN	...	4.0	10.0	0

5 rows × 30 columns

Task 1 - (level 2)

Data visualization

```
1 #filter to Look at the months where flights were cancelled
2 MonthsWithCancellations = dfData.loc[dfData['Cancelled'] == 1, 'Month'].unique()
3 print(MonthsWithCancellations)
```

[10 11 12]

```
1 import plotly.express as px
2 import pandas as pd
3
4 #define MonthRange from 1 to 12 to ensure all months are included
5 MonthRange = range(1, 13)
6
7 #reindex AmountCancellationsEachMonth to ensure all months are present, filling missing months with 0
8 AmountCancellationsEachMonth = AmountCancellationsEachMonth.reindex(MonthRange, fill_value=0)
9
10 #plot the data
11 fig = px.bar(x=list(MonthRange), y=AmountCancellationsEachMonth.values,
12             labels={'x': 'Month', 'y': 'Number of Cancelled Flights'},
13             title='Number of Canceled Flights Each Month')
14
15 fig.update_layout(
16     xaxis=dict(
17         title='Month',
18         tickmode='array',
19         tickvals=list(MonthRange),
20         ticktext=['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']),
21     yaxis=dict(title='Number of Cancelled Flights'),
22 )
23
24 #show the figure
25 fig.show()
26
```

Number of Canceled Flights Each Month

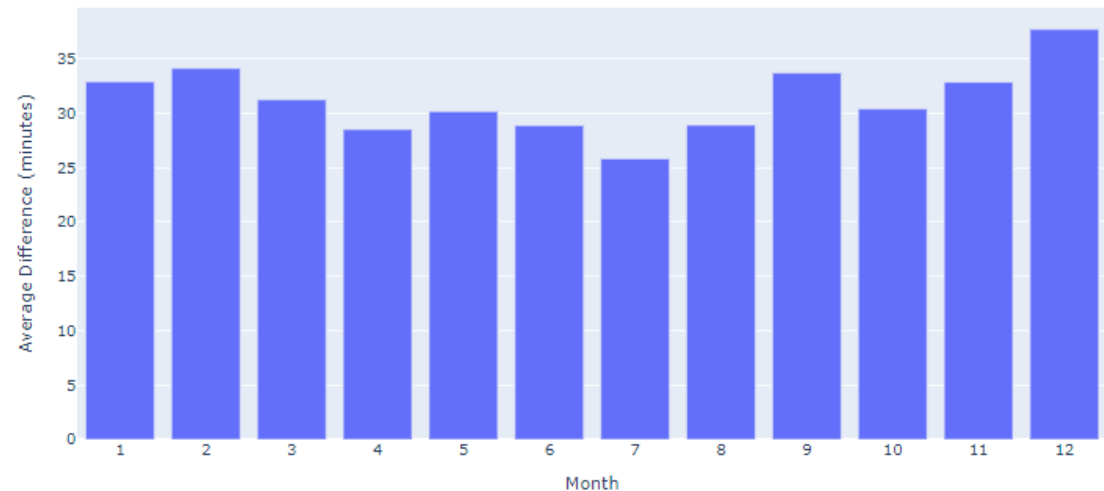


Task 1 - (level 2)

Data visualization

```
1 import plotly.express as px
2
3 #function to convert the values into minutes
4 def ConvertInMinutes(time):
5     hours = time // 100
6     minutes = time % 100
7     return hours * 60 + minutes
8
9 #convert the time into minutes
10 dfData['CRSDepTimeMinutes'] = dfData['CRSDepTime'].apply(ConvertInMinutes)
11 dfData['DepTimeMinutes'] = dfData['DepTime'].apply(ConvertInMinutes)
12
13 #calculate the difference
14 dfData['DepartureDifference'] = dfData['DepTimeMinutes'] - dfData['CRSDepTimeMinutes']
15
16 #group by month and calculate the average difference
17 AverageDifference = dfData.groupby('Month')['DepartureDifference'].mean().reset_index()
18
19 #create the bar chart
20 fig = px.bar(AverageDifference, x='Month', y='DepartureDifference',
21             title='Average Difference Between Scheduled and Actual Departure Time',
22             labels={'DepartureDifference': 'Average Difference (minutes)', 'Month': 'Month'})
23
24 fig.update_layout(
25     xaxis=dict(tickvals=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]),
26     bargap=0.2
27 )
28
29 #show the figure
30 fig.show()
```

Average Difference Between Scheduled and Actual Departure Time

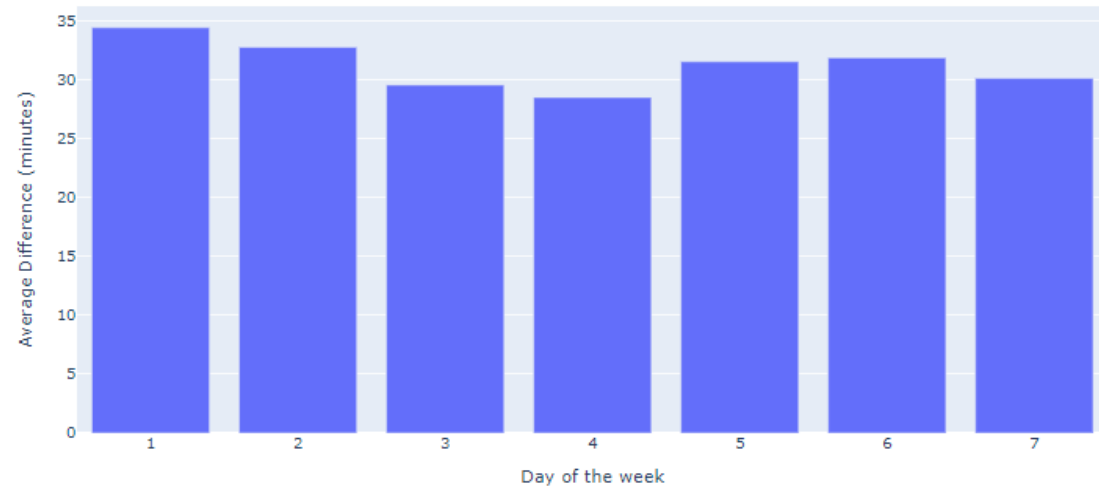


Task 1 - (level 2)

Data visualization

```
1 import plotly.express as px
2
3 #convert the time into minutes
4 dfData['CRSDepTimeMinutes'] = dfData['CRSDepTime'].apply(ConvertInMinutes)
5 dfData['DepTimeMinutes'] = dfData['DepTime'].apply(ConvertInMinutes)
6
7 #calculate the difference
8 dfData['DepartureDifference'] = dfData['DepTimeMinutes'] - dfData['CRSDepTimeMinutes']
9
10 #group by Month and calculate the average difference
11 AverageDifference = dfData.groupby('DayOfWeek')['DepartureDifference'].mean().reset_index()
12
13 #create the bar chart
14 fig = px.bar(AverageDifference, x='DayOfWeek', y='DepartureDifference',
15             title='Average Difference Between Scheduled and Actual Departure Time',
16             labels={'DepartureDifference': 'Average Difference (minutes)', 'DayOfWeek': 'Day of the week'})
17
18 #show the figure
19 fig.show()
20
```

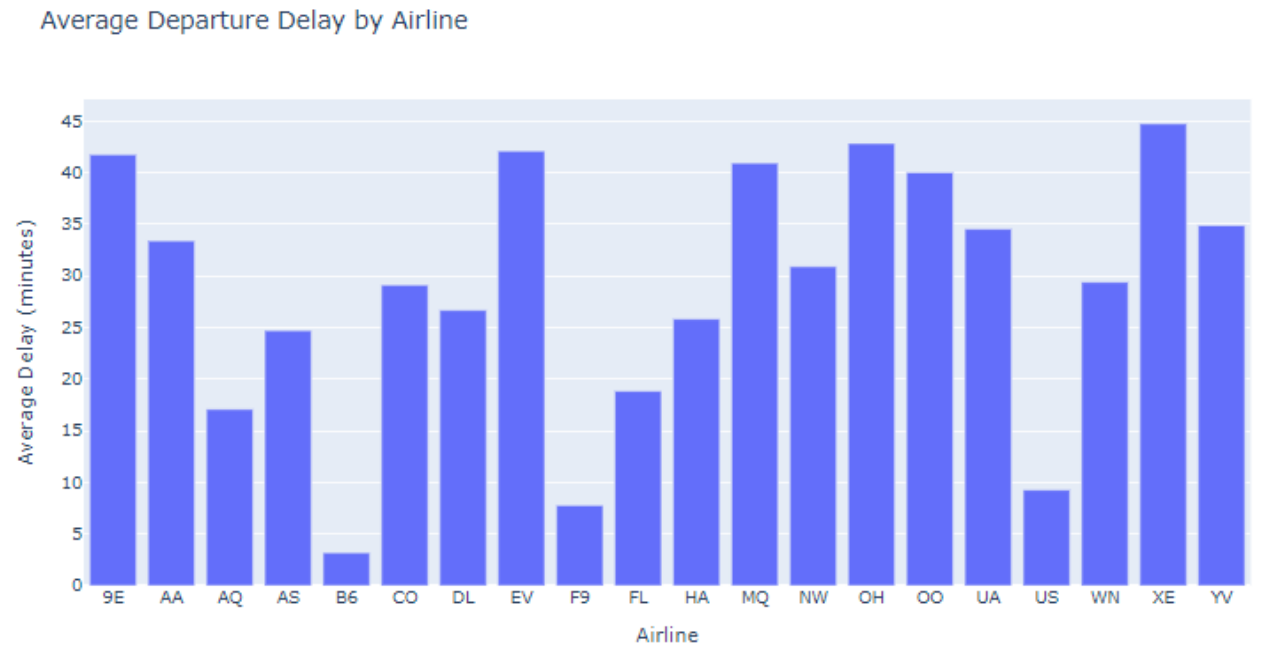
Average Difference Between Scheduled and Actual Departure Time



Task 1 - (level 2)

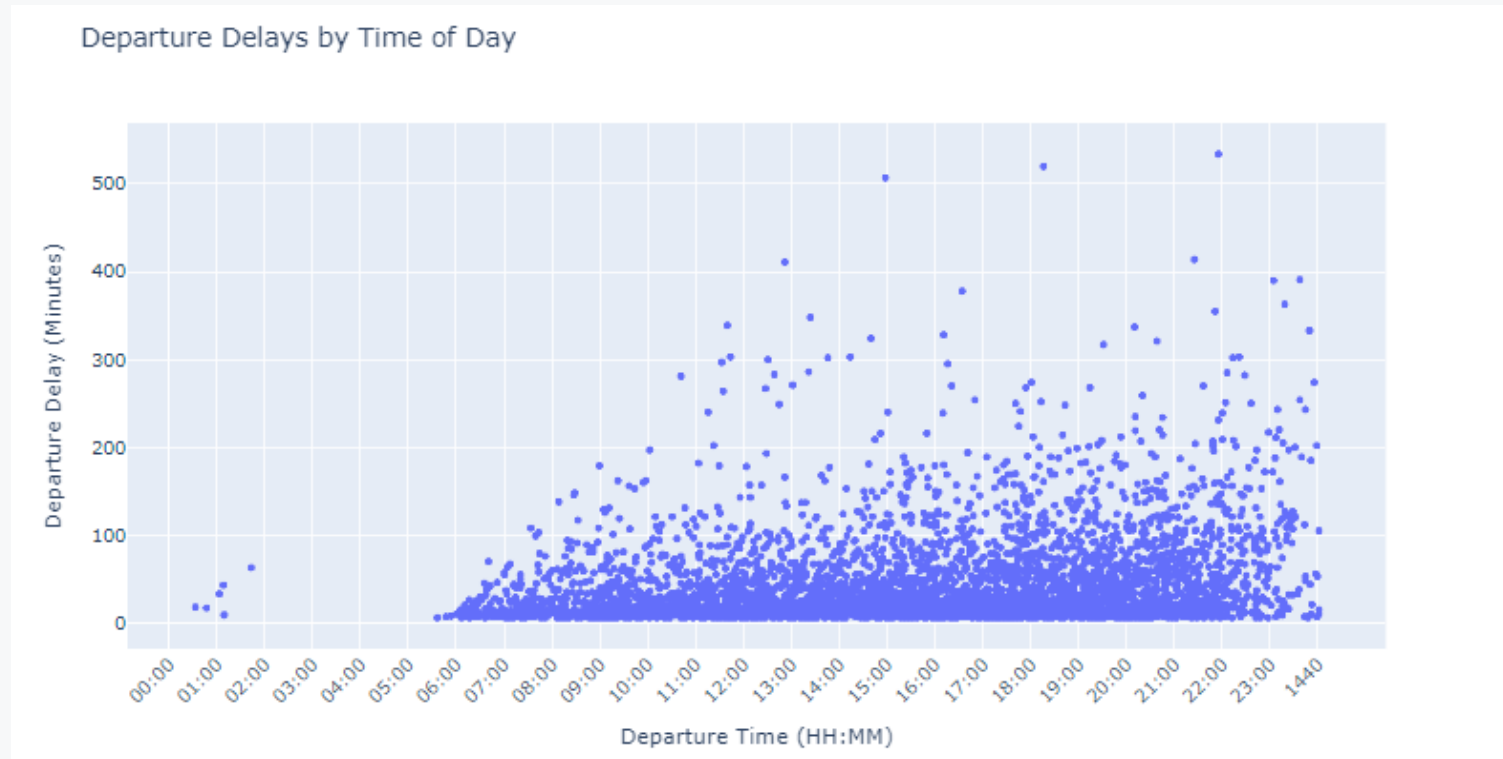
Data visualization

```
1 import plotly.express as px
2
3 #group by 'UniqueCarrier' and calculate the average delay
4 AverageDelayByAirline = dfData.groupby('UniqueCarrier')['DepartureDifference'].mean().reset_index()
5
6 #create bar chart for average delay by airline
7 fig = px.bar(AverageDelayByAirline, x='UniqueCarrier', y='DepartureDifference',
8             title='Average Departure Delay by Airline',
9             labels={'DepartureDifference': 'Average Delay (minutes)', 'UniqueCarrier': 'Airline'})
10
11 fig.update_layout(bargap=0.2)
12
13 #show the figure
14 fig.show()
15
```



Task 1 - (level 2)

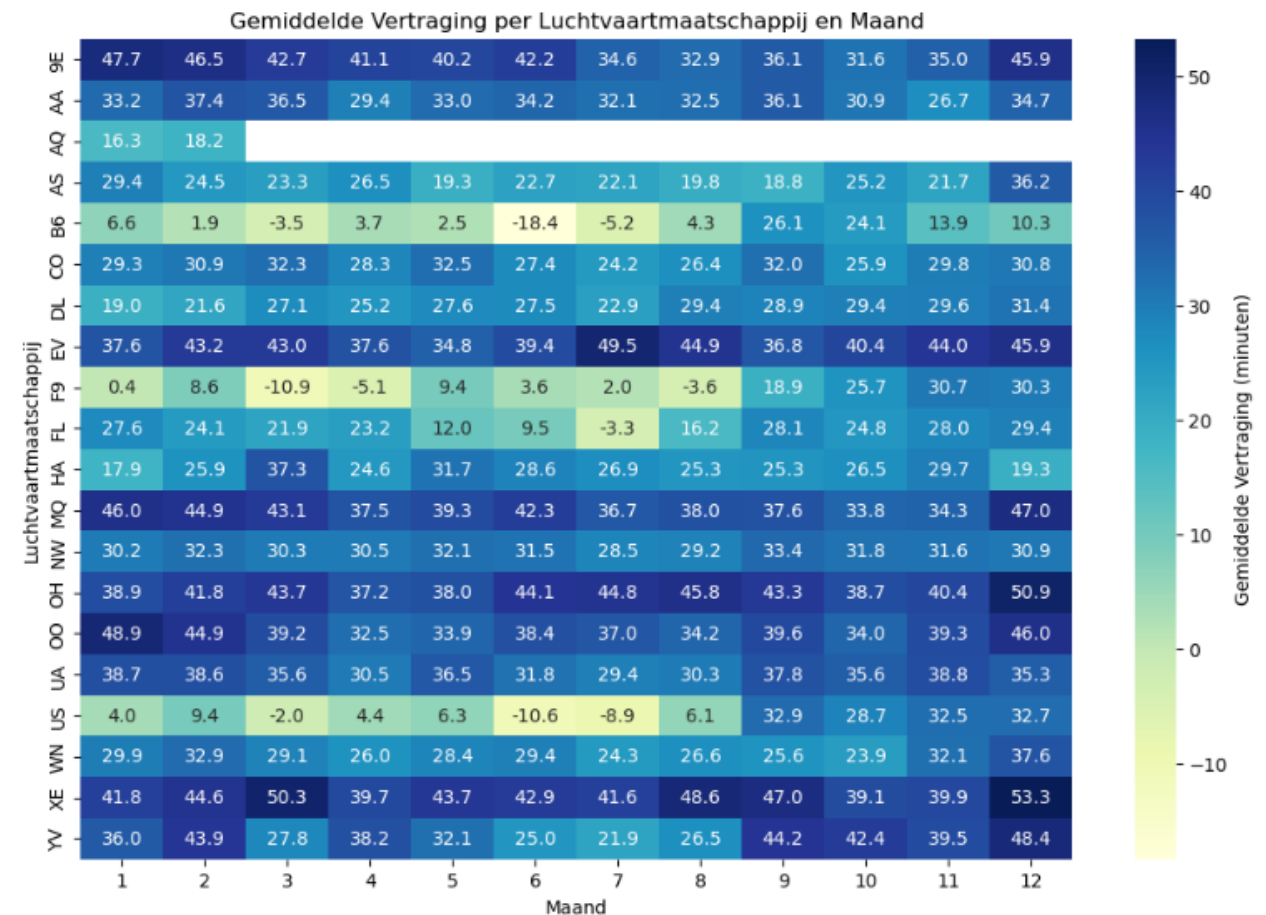
Data visualization



Task 1 - (level 2)

Data visualization

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 #bereken de gemiddelde vertraging per Luchtvaartmaatschappij en maand
5 heatmap_data = dfData.groupby(['UniqueCarrier', 'Month'])['DepartureDifference'].mean().unstack()
6
7 plt.figure(figsize=(12, 8))
8 sns.heatmap(heatmap_data, annot=True, fmt=".1f", cmap="YlGnBu", cbar_kws={'label': 'Gemiddelde Vertraging (minuten)'})
9 plt.title('Gemiddelde Vertraging per Luchtvaartmaatschappij en Maand')
10 plt.xlabel('Maand')
11 plt.ylabel('Luchtvaartmaatschappij')
12 plt.show()
13
```

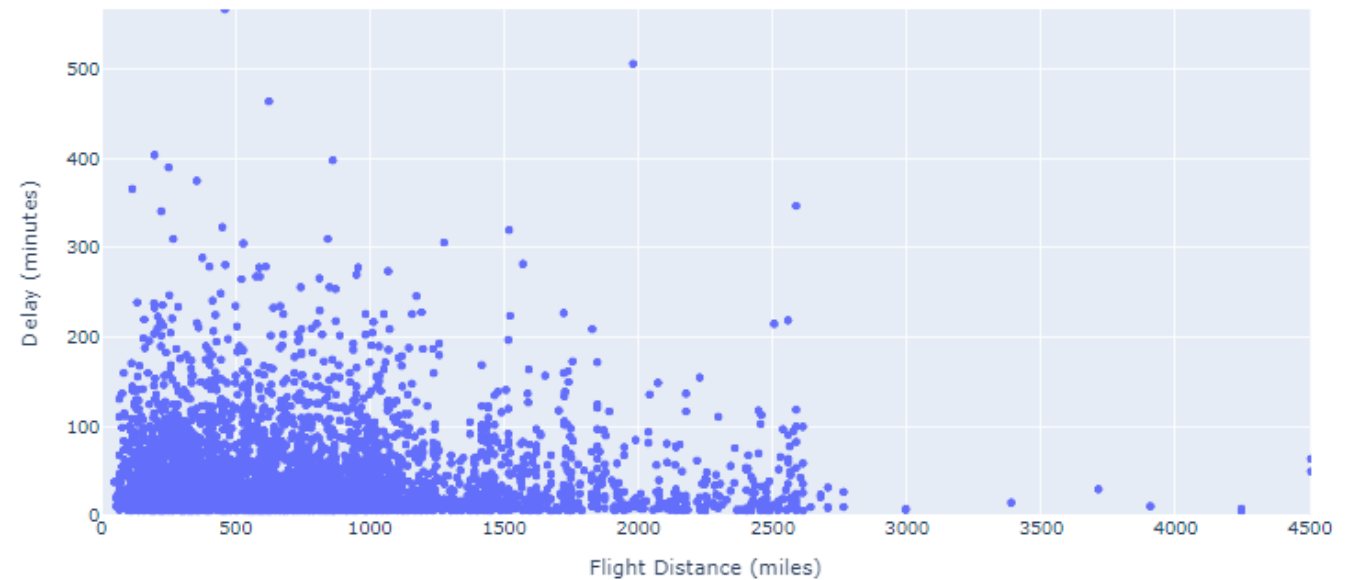


Task 1 - (level 2)

Data visualization

```
1 import plotly.express as px
2
3 #sample 5000 rows from the dataset for performance
4 sample_data = dfData.sample(n=5000, random_state=1)
5
6 fig = px.scatter(
7     sample_data,
8     x='Distance',
9     y='DepartureDifference',
10    title="Flight Distance vs. Delay",
11    labels={'Distance': 'Flight Distance (miles)', 'DepartureDifference': 'Delay (minutes)'}
12 )
13
14 # Update Layout to set the range for both axes
15 fig.update_layout(
16     xaxis=dict(range=[0, sample_data['Distance'].max()], title='Flight Distance (miles)'),
17     yaxis=dict(range=[0, sample_data['DepartureDifference'].max()], title='Delay (minutes)')
18 )
19
20 fig.show()
```

Flight Distance vs. Delay

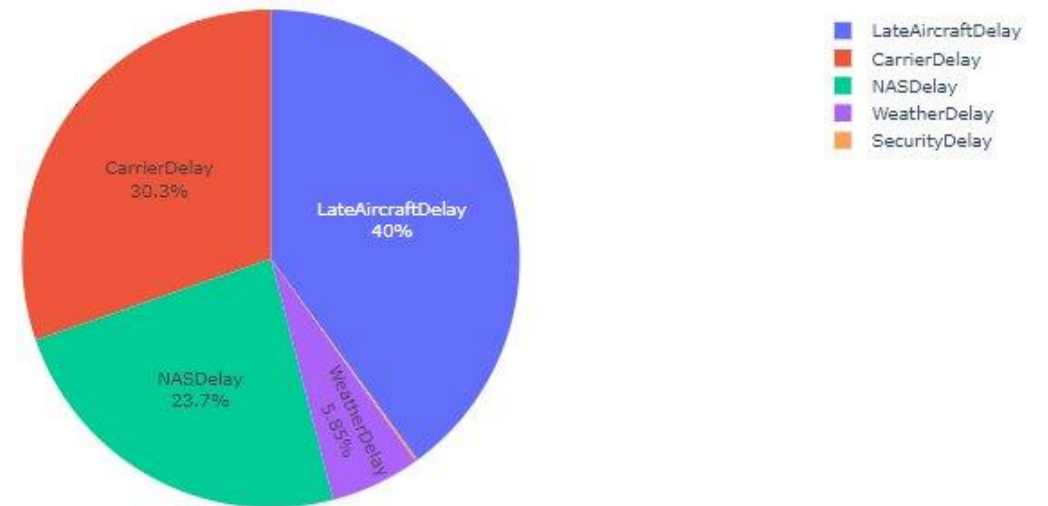


Task 1 - (level 2)

Data visualization

```
1 #summing each delay cause across all flights
2 delay_causes = dfData[['CarrierDelay', 'WeatherDelay', 'NASDelay', 'SecurityDelay', 'LateAircraftDelay']].sum()
3
4 #plot
5 fig = px.pie(values=delay_causes, names=delay_causes.index, title="Distribution of Delay Causes")
6 fig.update_traces(textposition='inside', textinfo='percent+label')
7
8 #show figure
9 fig.show()
```

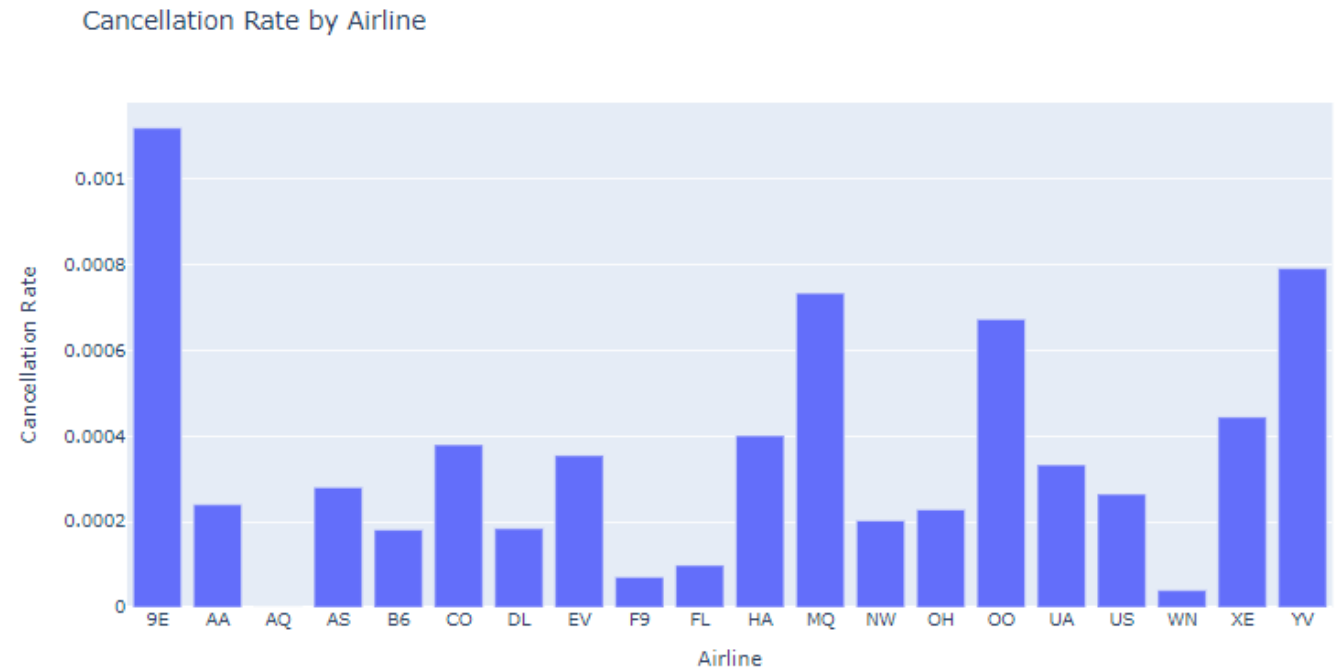
Distribution of Delay Causes

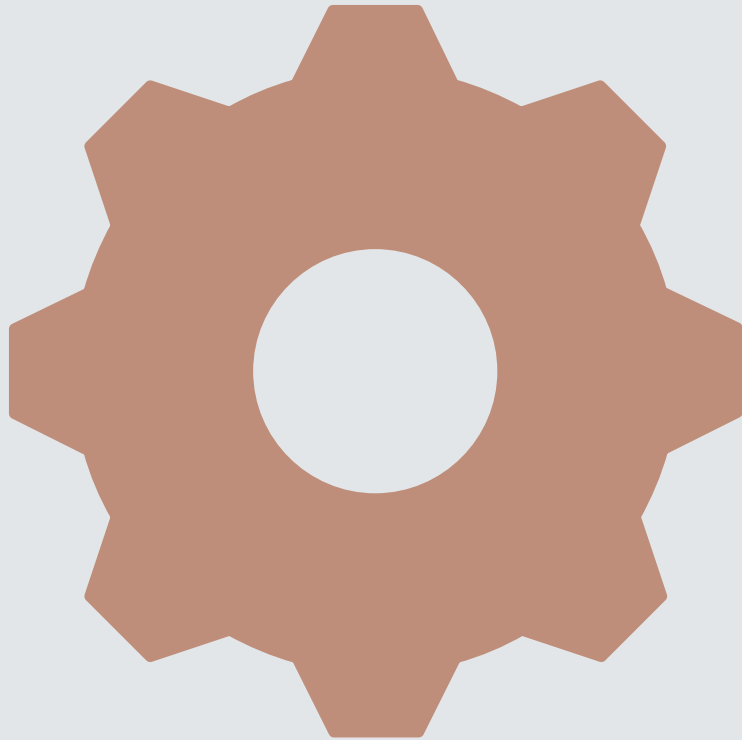


Task 1 - (level 2)

Data visualization

```
1 #calculate cancellation rate by airline
2 cancellation_rate = dfData.groupby('UniqueCarrier')['Cancelled'].mean().reset_index()
3
4 #plot
5 fig = px.bar(cancellation_rate, x='UniqueCarrier', y='Cancelled',
6              title='Cancellation Rate by Airline',
7              labels={'UniqueCarrier': 'Airline', 'Cancelled': 'Cancellation Rate'})
8 #show figure
9 fig.show()
```





TASK 1 - (LEVEL 2) DASHBOARD

QUESTIONS

